

UNIVERSITY OF ST ANDREWS

CS4302 COURSEWORK 2

Audio Processing

Author:
150008022

October 31, 2018

Description

The goal of this practical is to produce several PROMELA specifications that model a system and verify properties of the system.

Part 1

To model three processes, three proctypes were declared with similar semantics. They would each wait until the variable x corresponds to their process number, and that no other processes were still in their critical section.

```
1 bool s1 = false;
2 bool s2 = false;
3 bool s3 = false;
4
5 byte x = 1;
6
7 active proctype P1() {
8     x == 1 && !(s1 == true || s2 == true || s3 == true) ->
9     s1 = true;
10    x = 2;
11    /* Critical Section */
12    s1 = false
13 }
14 ...
```

To prove that this model of the algorithm described in the practical satisfied the mutual exclusion property, the following never statement was constructed:

```
1 never {
2     do
3         :: (s1 && (s2 || s3)) -> break;
4         :: (s2 && (s1 || s3)) -> break;
5         :: (s3 && (s2 || s1)) -> break;
6         :: else -> skip
7     od
8 }
```

The never statement would accept if it was possible for two or more of the states to be in their critical sections at any one time, which would therefore prove that this algorithm modelled as such does not fulfill the mutual exclusion property. The output of an exhaustive search shows that all processes reach all states, and that the never does not complete, therefore mutual exclusion is achieved with liveness conserved.

Part 2

For part 2 I found two approaches. The first uses a generalisation of the Peterson algorithm [1] for $N > 2$ processes, also known as the Filter algorithm for Mutual Exclusion [2]. The Filter algorithm uses a series of 'waiting rooms', which each process must pass through before being able to obtain the lock and progress through their critical section. This allows for priority to be given in a first-come first-serve basis. Processes are ensured to eventually reach the critical section, but the time this could take is not bounded [3].

In order to prove that no two processes would be 'printing' at the same time, a counter was also used alongside the print array. This counter would be incremented by processes after changing their print flag to true, and decremented afterwards. This allowed a simple never statement to be constructed that would only complete if the number of printing processes was ever greater than 2.

```
1 ...  
2 never {  
3     do  
4         :: numberPrinting > 1 -> break;  
5         :: else -> skip;  
6     od  
7 }
```

In order to try and further familiarise myself with PROMELA, I wanted to find a solution that utilised channels since they are a useful tool for modelling systems. A channel with a capacity of 1 holds the lock, which any process wishing to print must obtain. Processes poll the channel, and no two processes are able to remove the lock from the channel simultaneously. This provides a much simpler solution than the filter algorithm, and when verified with spin both solutions show that the never statements never accepts.

However, starvation is possible without weak fairness, as some processes may never get the chance to obtain the lock using the channel method.

```
1 ...
2 chan lockChannel = [1] of { bool }
3
4 active [N] proctype P() {
5     bool lock;
6     do
7         :: lockChannel?lock ->
8             /* Lock Acquired */
9             print[_pid] = true;
10            printing++;
11 progress:
12            /* Return Lock */
13            print[_pid] = false;
14            printing--;
15            lockChannel!true;
16        od
17 }
18 ...
```

Part 3

For the following questions, ltl formula was translated to never statements using spin's `-f` flag. In the code, the LTL used to generate the never statement is always included in a comment above the never statement.

3.a x is always odd

This statement is essentially already expressed in ltl, and so to produce the never statement only required a definition of odd, and the negation of the statement.

```
1 #define p (x % 2 != 0)
2 ...
3 never { /* !([] (p)) */
4 T0_init:
```

```

5      do
6      :: atomic { (! ((p))) -> assert(!(! ((p)))) }
7      :: (1) -> goto T0_init
8      od;
9  accept_all:
10     skip
11 }

```

pan:1: assertion violated !(!(((x%2)!=0)))(at depth 6) is the output of this verifier, therefore it is proven that x is not always odd.

Without weak fairness, it is possible that x would always be odd. If P2 and P3 were unable to run, then x would continue to be incremented to the next odd number, and even once overflow occurred with the byte value, it would still remain an odd number.

3.b It is possible that from a certain point onwards x is always odd.

LTL is not able to provide proof that a path exists between two states, unlike CTL [4]. However, we can prove it through reductio ad absurdum using LTL. Therefore we look for cases where odd is not eventually always odd by producing a never claim with the negation of that statement. i.e:

```

1  never {      /* not (eventually always (p)) */
2  T0_init:
3      do
4      :: (! ((p))) -> goto accept_S9
5      :: (1) -> goto T0_init
6      od;
7  accept_S9:
8      do
9      :: (1) -> goto T0_init
10     od;
11 }

```

Since the never claim enters an acceptance cycle, we have proof that it is possible that from a certain point onwards, x is always odd.

Without weak fairness however, the same answer as given in part a applies, as P2 and P3 are crucial to incurring even values of x .

3.c It is possible that from a certain point onwards x is infinitely often odd.

I understood this to mean that from any state onwards, there exists states where x is odd. This could be expressed as 'always eventually odd'. Negating this generated the following never statement:

```
1 never { /* not (eventually always (p)) */
2 T0_init:
3     do
4         :: (! ((p))) -> goto accept_S9
5         :: (1) -> goto T0_init
6     od;
7 accept_S9:
8     do
9         :: (1) -> goto T0_init
10    od;
11 }
```

When run through spin, the never claim enters an acceptance cycle, therefore we have proof that it is possible that from a certain point onwards, x is always odd.

Without weak fairness however, it is possible that P2 or P3 will cause x to be even, and then P1 will be the only process running from then on. This would cause x to remain even, as before, therefore this statement doesn't hold without weak fairness.

3.d It is always true that $y \leq x$.

This could be expressed in LTL as "always $y \leq x$ ". The negation produced the following never statement:

```
1 #define p (y <= x)
2 ...
3 never { /* not (always (p)) */
4 T0_init:
5     do
6         :: atomic { (! ((p))) -> assert(!(! ((p)))) }
7         :: (1) -> goto T0_init
8     od;
```

```

9  accept_all:
10      skip
11  }

```

When run, the assertion in the never claim is violated, therefore we have a case where y is greater than x . This is due to the eventual overflow of x .

Even without weak fairness, this statement still does not hold. While there are some paths where it may be true (eg. only P3 runs.), it would not be true for all paths.

3.e It is always true that when $y = x$ it follows that at some point $y \neq x$.

This can be expressed as:

```

1  never {    /* not (always ((y == x) -> (eventually (y != x)))) */
2  T0_init:
3      do
4          :: (! ((y != x)) && (y == x)) -> goto accept_S4
5          :: (1) -> goto T0_init
6      od;
7  accept_S4:
8      do
9          :: (! ((y != x))) -> goto accept_S4
10         od;
11  }

```

The output of this shows that the never claim never accepts. Therefore this statement is not true. Without weak fairness, this statement is also not true on all paths for similar to reasons to d).

Part 4

Underlying representation of PROMELA models

SPIN translates PROMELA into a finite non-deterministic automaton. Figure 1 shows this visually for a model of two concurrent processes. Specifically,

each process is an independent automaton, and each of their states are interleaved with those of the processes.

Verification is achieved by translating LTL into a Buchi automaton. The synchronous product of the claim and the automaton representing the processes (also referred to as the global state space) produces a Buchi automaton. This is why expressions in the special 'never' process interleave with the actual system processes. If the language of this resulting automaton is empty, then the claim is not satisfied, and otherwise, it will contain only the cases where the claim is true. This is utilised by spin to identify what series of system states will result in undesired behaviour.

The synchronous product of all of these automata is effectively a Cartesian product, which has an explosive growth rate. PROMELA places limits on certain resources such as number of processes, size of data types, etc, to ensure that the growth is restricted somewhat. This also ensures that any problem that can be described in PROMELA is decidable.

SPIN has been built for proving the negation of claims. Since most cases for models being tested will not suffer undesired behaviour, testing for all successful cases will require generating a far larger state space. By restricting it to those where a claim is not true, it is likely that the resulting state space will be far smaller.

LTL Model Checking Algorithms

SPIN uses Nested Depth-First Search to identify acceptance cycles. Each state will use two bits for each state to identify whether it is part of an acceptance cycle. This is proven by checking for a path between the root state of the system and the accepting state. The accepting state must also have a path to itself. Wherever both of these criteria are fulfilled, an acceptance cycle exists, which usually points to a case where the user defined claim does not hold. Weak fairness is included in this algorithm by assuming that any process with at least one possible transition eventually executes it.

Current Limitations

As mentioned in Part 3, LTL model checking does have limitations. It is only able to specify properties which are true for all paths. This means that it is unable to confirm the existence of paths between two states, which CTL is able to achieve.

Also, whilst the state explosion problem is handled very well in Spin, it is not completely annihilated, and some of the mechanisms used to reduce the search space (see next subsection) are lossy, therefore it is possible that some important paths will no longer be present.

Mechanisms for Improving Scalability and/or Efficiency

To further reduce the number of states required by spin to perform a verification, it uses partial order reduction. This essentially removes any series of state transitions that are representing two processes regardless of how the two are interleaved still have the same result. For example, the automata representing two processes carrying out operations on totally independent variables does not need to interleave all the states, as the processes don't affect each other. Utilising this results in a far smaller state space compared to standard searches, shown in Figure 2.

Spin also uses state compression in order to further reduce it's memory usage. It leverages the fact that processes and channels themselves have a very small statespace, and so instead of duplicating these everywhere, they can be created once and then referred to by an index elsewhere. For example, if a process has fewer than 256 states, then each state can be indexed using a single byte. Bit-state hashing is also used to further reduce the memory consumption. This involves using a hash function to produce an array index, and the value at this index indicates whether or not this state has already been seen [7].

- [1] G.L. Peterson. Myths about the mutual exclusion problem. *Information Processing Letters*, 12(3):115–116, 1981.
- [2] D.M. Hutton. The art of multiprocessor programming20083maurice herlihy and nir shavit. the art of multiprocessor programming. morgan kaufmann, 2008. 528 pp. price and further information: www.mkp.com, isbn: 978-0-12-370591-4. *Kybernetes*, 37(9/10):1583–1584, 2008.
- [3] Michel Raynal. Concurrent programming: Algorithms, principles, and foundations. *Springer Science & Business Media*, 2012.
- [4] Michael Huth and Mark D Ryan. *Logic in computer science*. Cambridge University Press, 2015.
- [5] Gerardo Schneider. Promela semantics lecture 8. 2007.
- [6] G.J. Holzmann. The model checker spin. *IEEE Transactions on Software Engineering*, 23(5):279–295, 1997.
- [7] Michael Westergaard. Hash compaction and bit-state hashing.



Figure 1: Process states represented as automaton. [5]

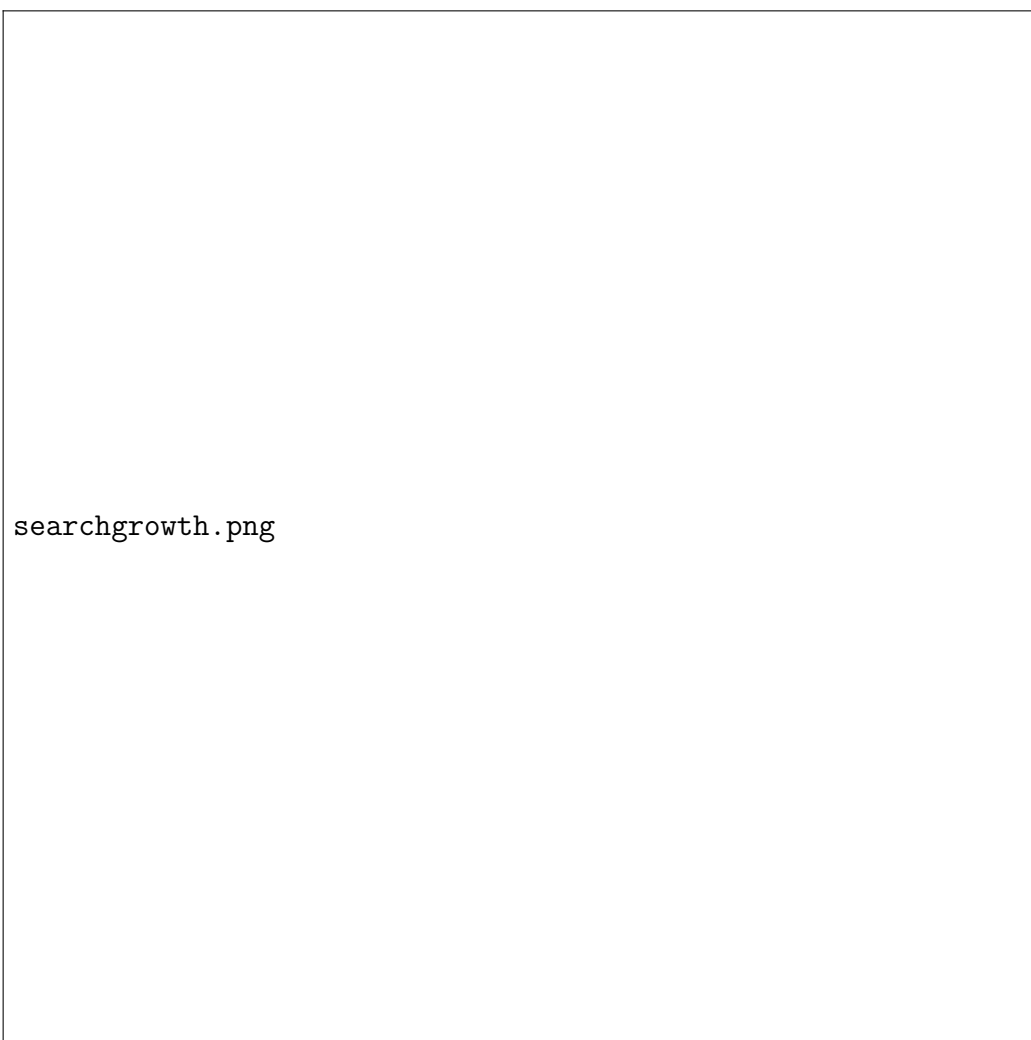


Figure 2: Effect of Partial Order Reduction. [6]